ALPEDIA ／ PHASE 1 REPORT

Alpedia

フェーズ1 開発完了報告
実装のまとめ・改善案・ローンチまでのロードマップ

物理演算付きボクセル・サンドボックス（Minecraft × Kerbal Space Program）

2026年10月　／　対象：Windows 版プロトタイプ（Python + Ursina/Panda3D）



PART 1 ／ ゲームについて

Alpedia とは ― 「作ったものが本当に動く」宇宙サンドボックス

ブロックが「やわらかい」

ボクセルは立方体の箱ではなく8頂点のポリゴン。素材ごとに硬さ・ちぎれやすさ・塑性が違い、金属・木・ゴム・粘土・液体まで同じ仕組みで表現。ぶつかれば曲がり、ちぎれ、別の構築物に分かれる。

惑星と軌道

球形の惑星・月・小惑星を重力で行き来。推進機とタンクでロケットを組み、重力ターンで軌道に乗せられる。遠点・近点や軌道速度を表示。

機器とプログラム

推進機・タンク・配管・発電機・バッテリー・アクチュエーター・アンテナなど15種。コンピューターに Python かブロックプログラムを入れて、ロボットやドローンを自動制御。

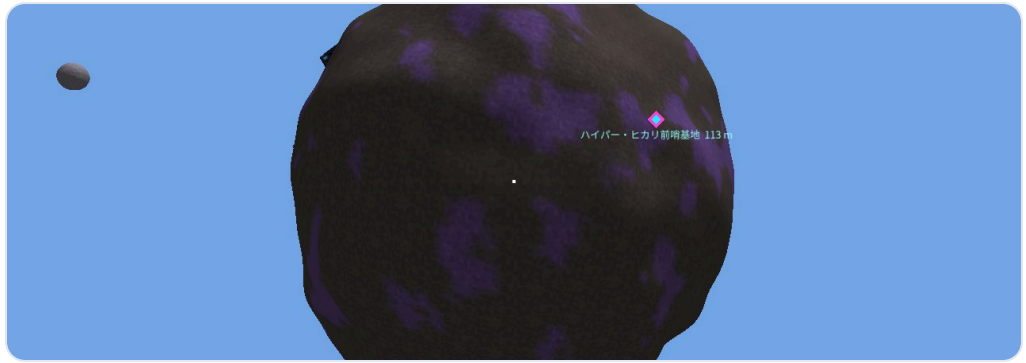
暮らしと世界

サバイバルでは採掘・精錬・燃料作り。地表や軌道にはサイバーパンクな街があり、取引や部屋の契約ができ、都市どうしは征服や攻撃をし合う。

設計方針：描画は必要最低限、遊びの幅は「物理 × 機器 × プログラム」の組み合わせで作る。素材・機器・惑星は MOD フォルダのテキストで追加でき、構築物は OBJ ファイルで共有できる。



惑星ガイアの丘に生成されたサイバーパンクの街



星鉱石の採れる小惑星（重力 2.5 m/s²）

PART 1 ／ 実装の流れ

フェーズ1 の開発の流れ

「物理で動くボクセル」の芯を先に作り、機器 → プログラム → 世界 → 暮らし → 配布の順に層を重ねた。

① 物理の芯

- 8頂点ボクセルと PBD（形合わせ＋結合）
- 素材ごとの硬さ・破断・塑性・再結合
- 丸いブロックの形（球＋結合部）と当たり判定
- 惑星の重力・地形・水と浮力

② 機器と制御

- 推進機・燃料・配管・電力・気密
- アクチュエーター（回転・押し出し・分離・結合）
- ゲーム内 Python とブロックプログラム
- OBJ 書き出し・読み込み、MOD

③ 軽量化

- 剛体化・遠方のレール・大型のチャンク化
- numba で機械語化＋複数コア
- 完全剛体モード（ワールド設定）
- 関節つき剛体（アクチュエーター前後）

④ 世界と暮らし

- 月・小惑星、資源（硫黄・星鉱石）
- サバイバル・スターターキット
- サイバーパンクの街（地表・軌道）
- 通貨・NPC・部屋、都市の思惑

⑤ 仕上げと配布

- チュートリアル（姿勢制御・ロケット・ドローン）
- 無線（アンテナ）と遠隔操作
- GPU シェーダーによる描画の高速化
- Windows 用ワンクリックパッケージ

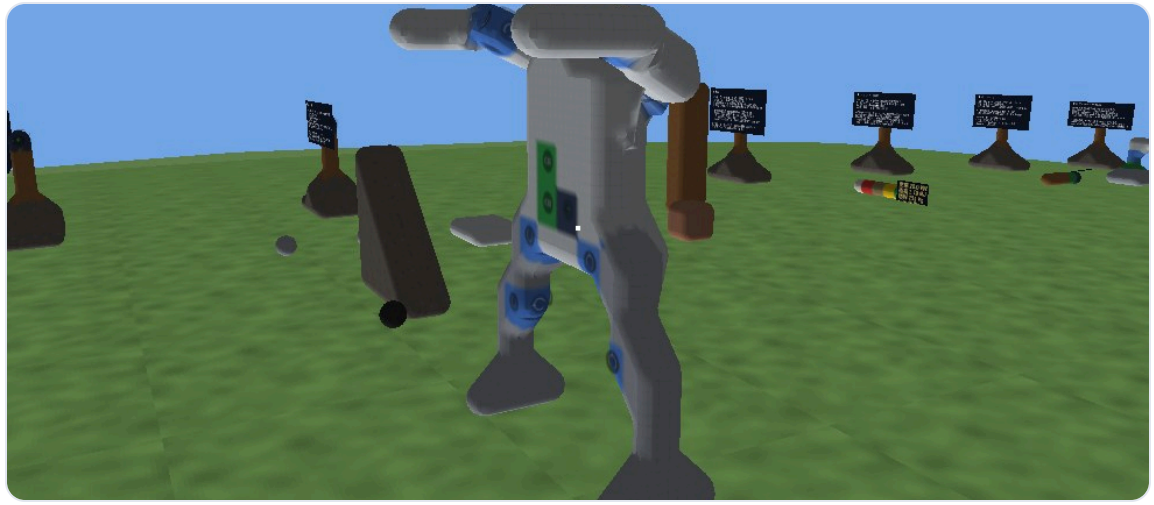
● 品質面での主な修正（後半）

3000 秒遊ぶと重くなる問題（メニューを閉じても部品が消えず毎フレームの処理が増え続ける）を修正し、画面を開け閉めしても部品数が増えないことを確認。マーカーの左右反転、OneDrive 上でのワールド削除エラー、F10 キーの競合（F8 へ移動）なども解消。

物理エンジン — やわらかいボクセルと、場面に応じた計算の切り替え

計算のしかた	いつ使うか	特徴
ブロックごと（PBD）	変形・衝突・破断が起きている構築物	形合わせ＋結合。曲がる・ちぎれる・再結合
剛体	形が変わらず落ち着いている構築物	1つの剛体として回転・並進だけ計算
レール	プレイヤーから遠い構築物	重心の軌道と回転だけ（数値積分）
チャンク	300 ブロック以上の大型	4×4×4 ごとに剛体、境目だけ結合
完全剛体（ワールド設定）	常に	壊れない。接地は剛体として解く（NNLS）
関節つき剛体	完全剛体でアクチュエーターが動く間	部分ごとの剛体を関節でつなぐ（XPBD 剛体版）

重い部分（形合わせ・結合・当たり判定・関節ソルバー）は numba で機械語化し、構築物ごとに複数の CPU コアで並列に計算。



関節つき剛体の検証用ロボット（69ブロック・関節9つ）。腕を上げて手を振る

1.3 mm
手を振る動作の関節のずれ（中央値）

0 個
関節つき剛体で壊れた部品（同じ動作はブロックごとの計算だと 4〜8 個に分解）

0.55°
チュートリアルの姿勢制御の棒の誤差

機器 — 配線いらずの電力・制御と、つながった配管

動力・推進

- **推進機**：燃料を使って押す（150 kN）。スロットル・割り当てキー
- **タンク・配管・バルブ**：連続したボクセルで流体がつながる
- **発電機・太陽光パネル・手回し発電機**
- **バッテリー**：同じ構築物の中で電力を共有

動く・作る

- **アクチュエーター**：回転／押し出し／引き込み／分離／結合。P で設定、E で実行
- **精錬機・加工機**：鉱石→鉄・シリカ、硫黄・水→燃料、鉄→部品
- **通気口**：閉じた空洞を自動検出して与圧・減圧

考える・伝える

- **コンピューター**：Python／ブロックプログラムを実行。表面に文字を表示（看板）
- **操縦席**：座っている間のキー入力をプログラムへ
- **アンテナ**（新）：届く範囲 3000 m。構築物どうしで無線通信

しくみの約束ごと

電力と信号は **同じ構築物の中なら配線不要** で共有。配管だけは **連続したボクセル** が必要。アクチュエーターで分かれた部分どうしでも、電力・信号・配管は今までどおり通る。

MOD で増やせる

機器・素材・惑星・プレイヤーの定義は JSON。`mods/` に置くだけで性能や見た目の違う派生を追加でき、基本セットはあえて「上位版なし」の最小構成にしている。

プログラミング — Python とブロック、そして無線

ゲーム内 Python

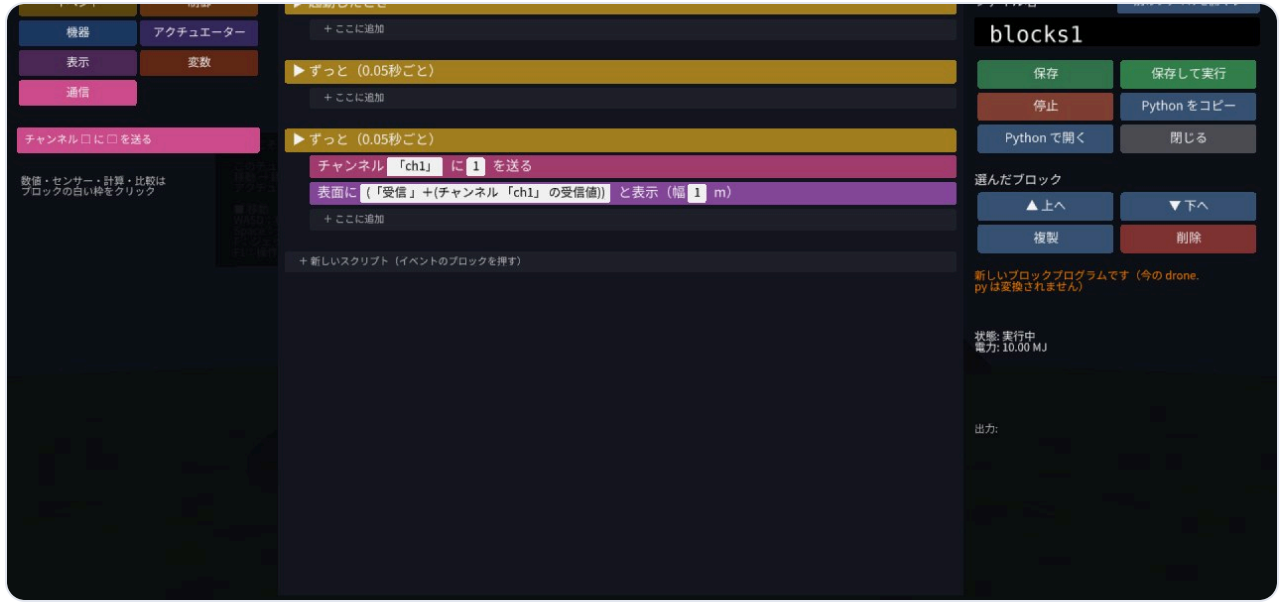
コンピューターに `tick(api)` を書くと 0.05 秒ごとに実行。高度・速度・姿勢・軌道・電力を読み、推進機やアクチュエーターを動かす。使えるモジュールは math / random / statistics の簡易サンドボックス。

ブロックプログラム

イベント・制御・機器・アクチュエーター・表示・変数・通信のブロックを並べるだけ。保存すると Python に変換されるので、上達したら「Python をコピー」で書き換えへ移れる。

無線通信（アンテナが必要）

`api.send(ch, 情報)`／`api.receive(ch)`／`api.latest(ch)`。届く距離は両方のアンテナの短いほう、情報は JSON にできる値。アンテナが無い・OFF・電力不足ならエラーになる。



ブロックエディタに追加した「通信」ブロック（送信と受信値）



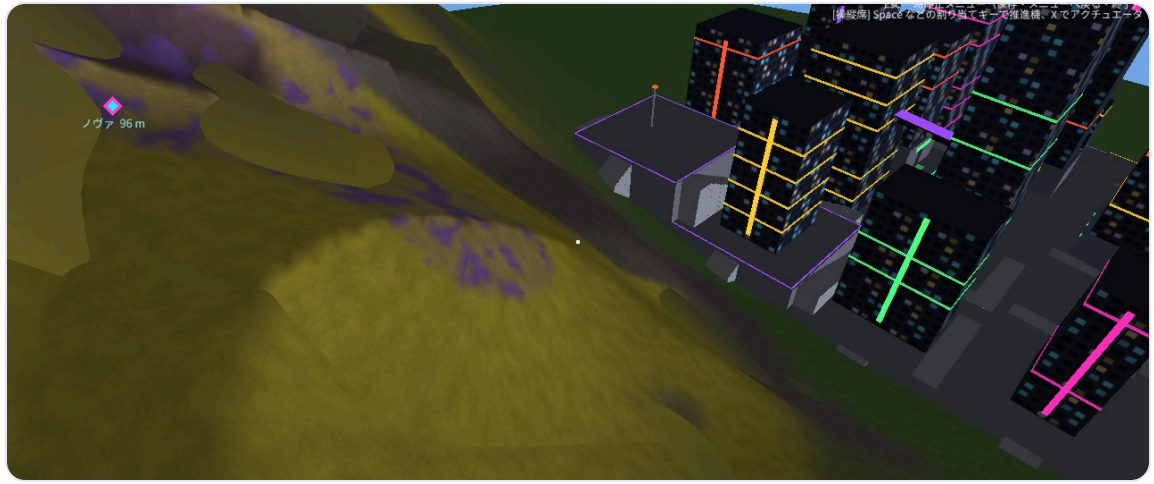
ドローン操縦台の画面：ドローンから無線で届いた高度・速さ・燃料を表示

世界 — 惑星・月・小惑星と、資源のある地形

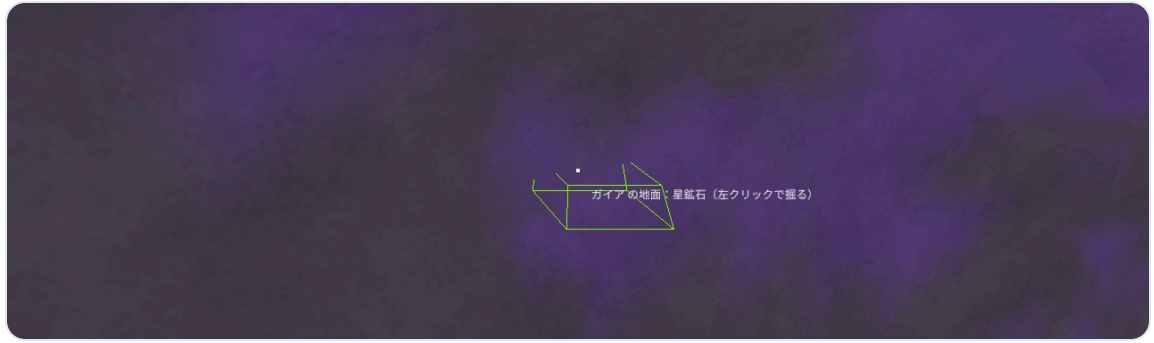
天体	特徴
テラ	出発点。平地・山・海。山には硫黄鉱石の鉱脈
ガイア	起伏の大きい惑星。洞窟の奥で星鉱石
ルナ（月）	低重力。掘ると星鉱石
小惑星 ×3	6〜8 km 先。重力 2.5 m/s^2、星鉱石が採れる

地形と描画

- 惑星は距離に応じた細かさの地形パッチ＋掘った所だけ細かいメッシュ
- 浮動原点：すべてカメラからの相対座標で描くので、世界が広くても精度が落ちない
- 水はクリックで汲める（水面は減らない）。精錬すると燃料に



山あいの街と硫黄鉱石の鉱脈（黄色い斜面）



洞窟の奥の星鉱石（紫）

サバイバル — 掘る・精錬する・燃料を作る・稼ぐ

● 詰まらない始まり

- スターターキット（鉄 250・シリカ 40・部品 12・燃料 60）
- 置いたバッテリーは空から始まる
- ジェットパックは手持ちの燃料を使う

● 燃料の作り方

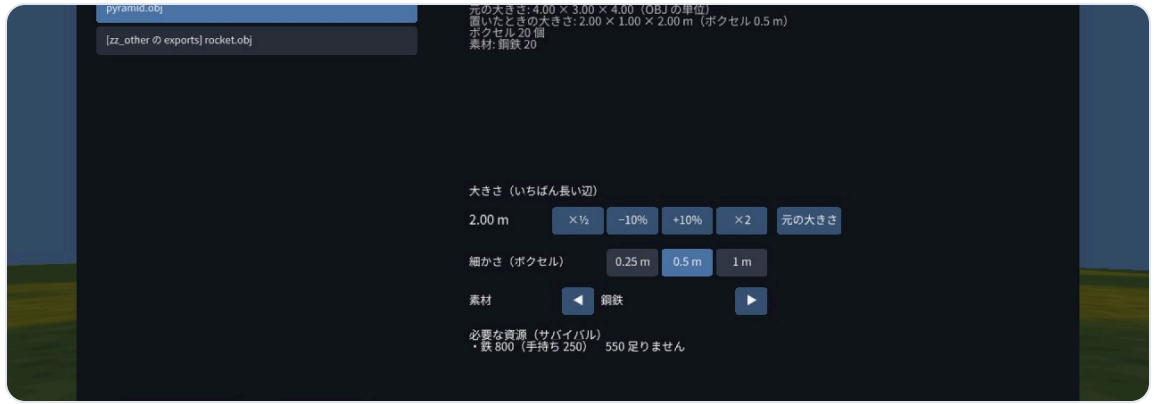
- 硫黄鉱石 10 → 燃料 20（主な手段）
- 水 10 → 燃料 5（電力を多く使う）
- E でインベントリからタンクへ給油
- 燃料発電機は電気の使い道があるときだけ燃やす

● 建てる・読み込む

- ブロックや機器は資材を消費
- F8 の読み込みメニュー：置くもの・大きさ・必要な資材を確かめてから配置
- OBJ で他の人の構築物を取り込める

甘めの経済バランス

鉱石を買って精錬して売るだけでも、どの地域でも **+7〜39%** の利益。買値は基準の 1.1 倍、売値は 0.9 倍。地域ごとに相場が違う（火山の街は硫黄が安い、海辺は燃料が安い、軌道上は鉱石が安く燃料がとても高い）ので、いろいろな街へ行く理由になる。



F8 の読み込みメニュー（大きさの指定と必要な資材）

サイバーパンクの街 — 暮らし・取引・都市どうしの争い

街の生成

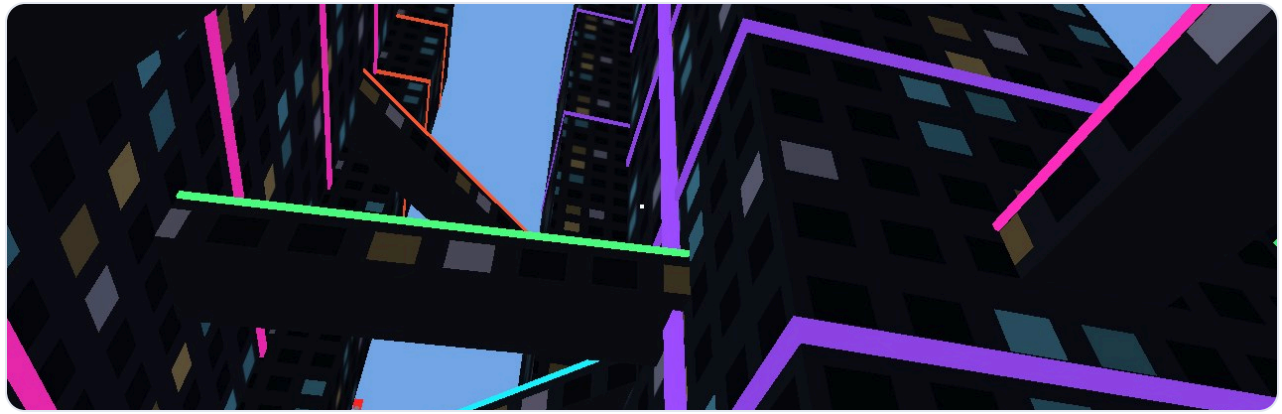
- 地表は山あい・海沿いにも 520 m 間隔、軌道上はステーション
- 15 m のテラスと階段で歩きやすい舗装（階段の登りテスト 36/36 成功）
- ぶつかると壊れた見た目に。軌道上の街は押す・推進機で軌道が変わる
- 100 m より遠い街は簡略表示（街の中 43.6→26.5 ms/フレーム）

暮らし

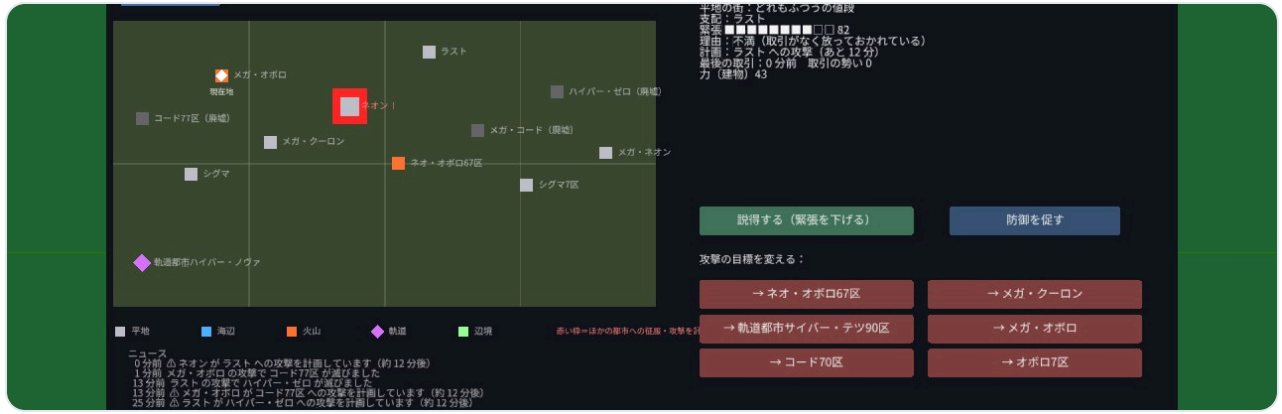
- 通貨・NPC（商人・不動産屋・住民）・内装のある建物
- 部屋を買う／借りる、収納箱、ベッド。M キーで街・自宅の目印

都市の思惑（K キーのマップ）

- 取引が多すぎると「野心」、放っておくと「不満」がたまる
- 緊張が 70 を超えると、約 12 分後に征服か攻撃。滅びた街は廃墟に
- プレイヤーは通信費を払って「説得」「防御を促す」「目標を変える」



街の中（ネオンの縁取りと窓）



訪れた都市のマップと、都市への助言

チュートリアル — 看板に沿って、遊びながら覚える

順番

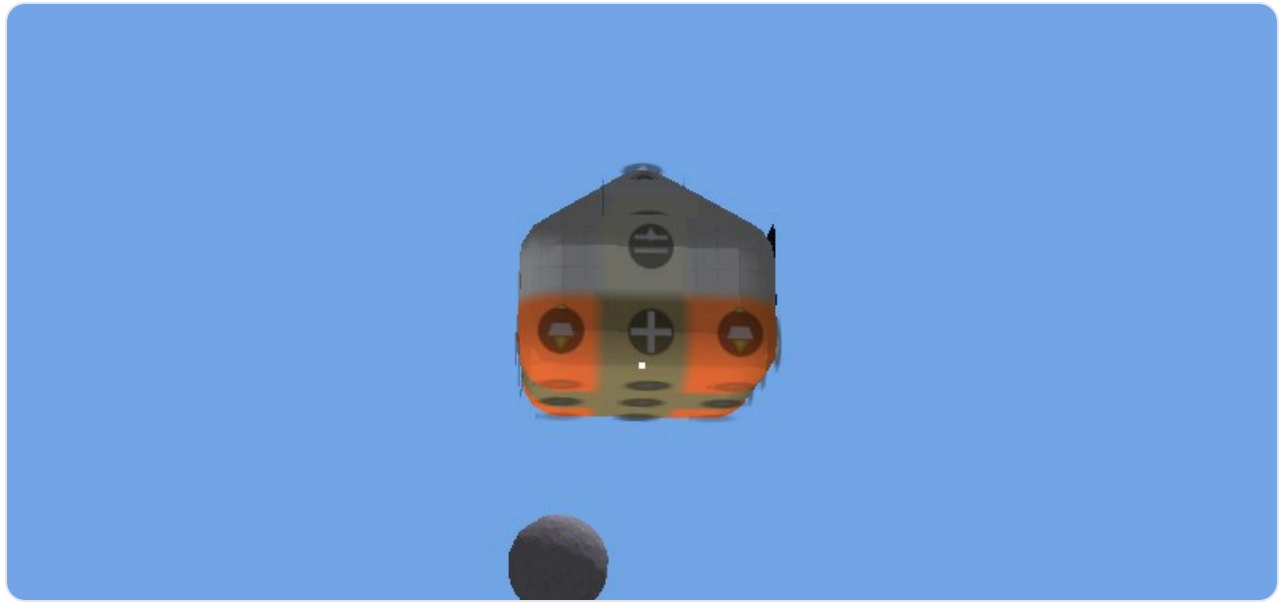
移動 → 建築 → ブロックの形 → 物理 → 機器 → コンピューターと看板 → アクチュエーター → **棒の姿勢制御** → **ロケットと軌道投入** → **無線とドローン**

棒の姿勢制御

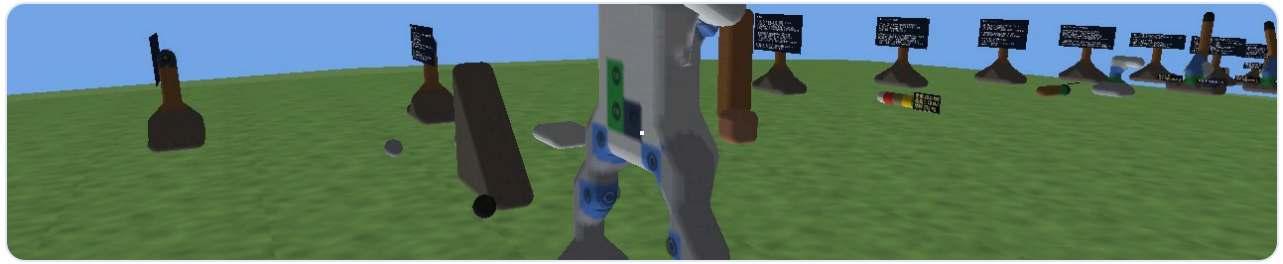
揺れ続ける台の上の棒を、ブロック数個のプログラムで真上に保つ。お手本と課題の装置を並べて置き、看板の手順どおりに並べると完成する。

ドローンの遠隔操作（新）

操縦台に座ると、押したキーが無線でドローンへ。W/S/A/D で前後左右、Space/Shift で上下、X で着陸・離陸。電波が切れると自動で着陸する。前後左右は約 8 m/s、高さのぶれは ±5 cm 以内。



操縦台から見たドローン（推進機の噴射はオレンジで表示）



完全剛体モードで動くロボット

技術基盤・高速化・ワンクリック配布

層	使っているもの
言語	Python 3.12（ゲームもスクリプトも）
描画・入力	Ursina 8.3 / Panda3D 1.10（Ursina 本体は改変なし）
数値計算	numpy・scipy・scikit-image、numba（機械語化・並列）
GPU	GLSL：コンピュートシェーダーで頂点位置、頂点シェーダーで法線、陰影はピクセルごと
配布	Python 本体とライブラリを同梱。C 言語のランチャーを zig でクロスコンパイル

2.67→0.14
ms/フレーム：やわらかい構築物の描画更新（GPU 化）

43.6→26.5
ms/フレーム：街の中の描画（遠い街は簡略表示）

増えない
画面を開け閉めしたあとの部品数（前は開くたびに数十個残り続けた）



上：CPU 計算、下：GPU 計算。見た目は同じで、CPU の負担は約 1/20

パッケージ版は `Alpedia.exe` をダブルクリックするだけで起動。元のフォルダやセーブとは別に保存し、ゲームを直したら「パッケージを作る.bat」で作り直す。

PART 2 ／ 改善案

改善案の全体像 — 3本の柱

プロトタイプで「遊びの芯」は確かめられた。次は **どこでも動く・一緒に遊べる・分かりやすく気持ちいい** の3点を固める。

① 軽量化とマルチプラットフォーム

今の課題

- Python の1本のスレッドで、ゲームの進行・UI・描画準備を全部回している
- 物理は CPU（numba）。ブラウザ・スマホ・家庭用ゲーム機では動かせない
- 描画が Ursina/Panda3D に結びついている

方向：シミュレーションの核を描画から切り離し、どの描画エンジンからも使える形にする

② マルチプレイヤー化

今の課題

- ワールドは1台の中だけ。保存も1人分
- やわらかい物理は計算量が多く、状態を丸ごと送ると通信量が大きい
- ゲーム内 Python をどこで動かすか（安全性）

方向：サーバーが正しい状態を持つ形を軸に、遊び方に合わせて複数の接続方式を用意する

③ ゲーム性の向上

今の課題

- ブロックの境目がぼやけて見え、形が読み取りにくい
- UI が画面ごとにばらばらで、キー操作が多い
- チュートリアルが看板の説明中心で、目的が弱い

方向：くっきりした見た目、統一した UI、物語に沿ったチュートリアルと目標

シミュレーションの核を独立させ、描画は差し替えられる形に

3層に分ける

核：物理・機器・世界・通信（描画なし、決まった刻みで進む）

橋渡し：状態の差分・メッシュ・イベントを出す共通の形式（C ABI / WASM）

表示：描画エンジン・UI・入力（プラットフォームごとに選べる）

今の Python 版も「核」と「表示」の境目（world ↔ client）がほぼ分かれているので、段階的に移せる。

中身の軽量化（どの選択でも効く）

- 物理の GPU 化：形合わせ・結合をコンピュートシェーダーへ（WebGPU / Vulkan / Metal で共通）
- 剛体・関節つき剛体を標準にし、ブロックごとの計算は「壊れる瞬間」だけにする
- 街・地形はインスタンス描画と距離による段階表示を徹底
- 決まった刻み・決まった順の計算にして、マルチプレイと再現テストに使う

選択肢	良い点	気になる点
A. 今の Python 版を磨く	すぐ進められる。MOD・スクリプトと相性が良い	Web・スマホ・家庭用ゲーム機は難しい。GIL で並列化に限界
B. 核を Rust（または C++）で書き直す	ネイティブと WASM の両方に出せる。速くて並列にしやすい	書き直しの工数。スクリプトの仕組みを作り直す
C. 汎用ゲームエンジンへ移す（Godot / Unity / Unreal / Bevy）	各プラットフォームへの書き出し・UI・音が揃っている	エンジンへの依存、ライセンス、特殊な物理は結局自前

提案

B の核（Rust → ネイティブ／WASM）＋ **表示は差し替え可能**。まず PC 向けは今の描画のまま核だけ入れ替え、Web 版は WebGPU、家庭用ゲーム機やスマホは C の汎用ゲームエンジンを表示に使う。ゲーム内スクリプトは Python 風の言語（または WASM サンドボックス）で互換を保つ。

接続方式の比較 — 遊び方ごとに向き不向きがある

方式	しくみ	運営費	ずる対策	遅延	人数・世界の大きさ	物理の同期	向いている遊び方
専用サーバー	運営側のサーバーが正しい状態を持つ	高い	強い	中	大人数・常設の世界	サーバーで一括	公開サーバー、街・経済を共有
ホスト型（リッスンサーバー）	遊ぶ人の1台がサーバーを兼ねる	ほぼ無し	ホスト次第	中	数人〜十数人	ホストで一括	友だちと協力
中継つき（リレー／NAT 越え）	ホスト型の通信を中継サーバー経由で届ける	低い	ホスト次第	中〜やや大	数人〜十数人	ホストで一括	ルーター設定なしで遊ぶ
P2P（入力だけ送る）	全員が同じ計算をして入力だけ交換	ほぼ無し	弱い	小（巻き戻しで隠す）	少人数	完全に同じ計算が必要	対戦・レース
アドホック（LAN・Wi-Fi Direct 等）	近くの端末どうしで直接つなぐ	無し	身内向け	小	2〜8人程度	ホスト端末で一括	同じ部屋・ネット無しの場所

提案：同じ「サーバーが正しい状態を持つ」作りで3つの入口

ゲームの中身はどれも同じサーバー権限型にして、**①ホスト型（LAN／アドホック含む）→ ②中継つき → ③専用サーバー** の順に広げる。サーバーの核は柱1の共通の核をそのまま使う。

Alpedia ならではの工夫

- 送るのは頂点ではなく「部分ごとの剛体の姿勢」と変形の差分（関節つき剛体が効く）
- 遠い構築物はレールのまま、近い所だけ細かく（関心範囲で間引く）
- ゲーム内スクリプトはサーバー側のサンドボックスで実行、無線もサーバーが仲介

くっきりした見た目・迷わない UI・物語のあるチュートリアル

見た目をパキっと

- 面ごとに正しい UV（今は箱型の投影で、境目でテクスチャが伸びてぼやける）
- 角の丸みは小さく（面取り程度）にし、球と結合部の形は「壊れたとき」「やわらかい素材」だけに
- ピクセルアート調のテクスチャ集をくっきり表示（近くは拡大しても滲ませない）
- ブロックの縁の線・軽い環境光の陰（AO）で形を読みやすく
- 素材ごとの色の決まりと、機器の記号の統一

迷わない UI

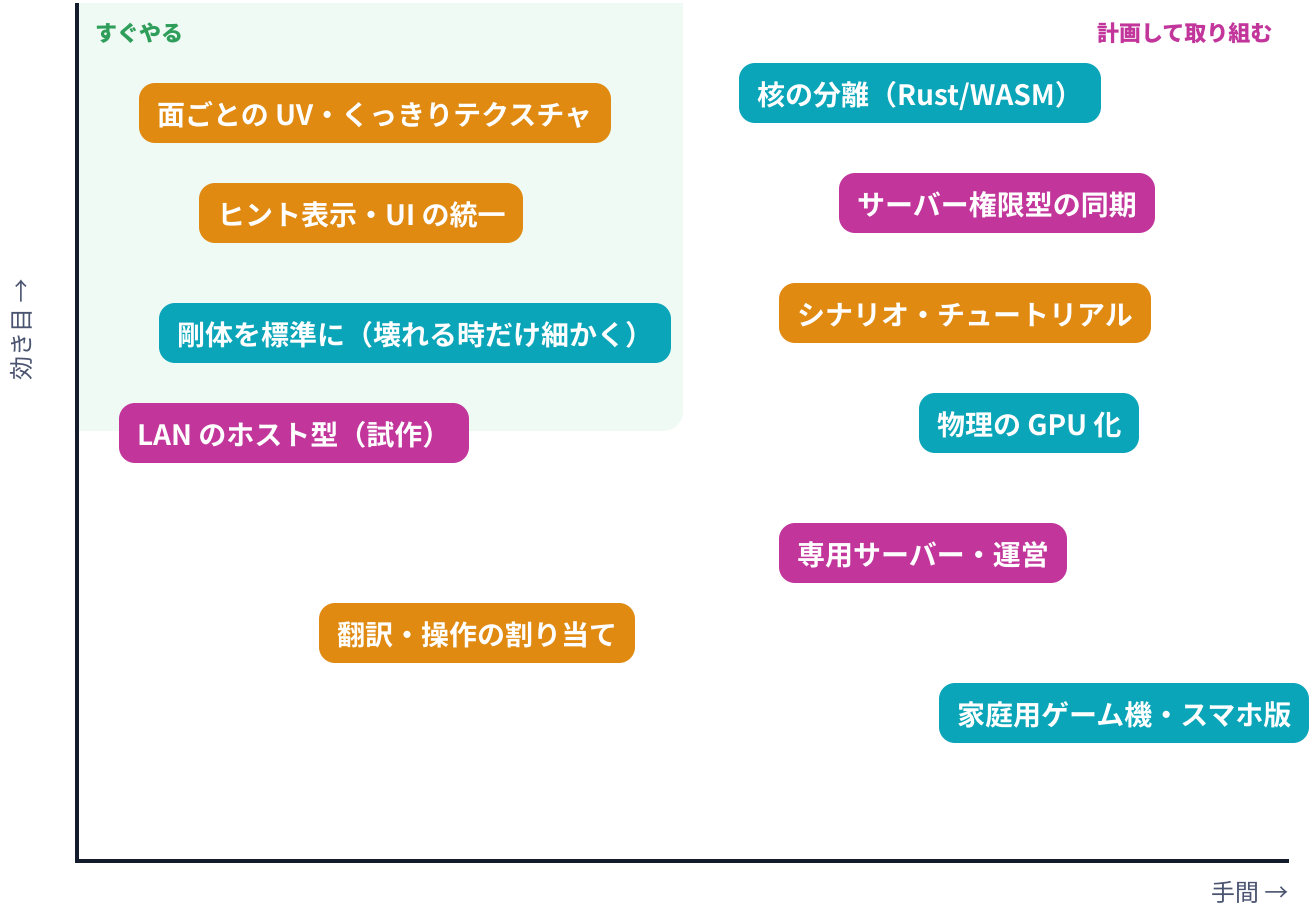
- 画面ごとにばらばらな作りを、1つのデザインの決まり（色・余白・ボタン）にそろえる
- よく使う操作は円形メニュー（長押し）にまとめ、キーの数を減らす
- 狙った物の説明と「今押せるキー」を画面下に常に表示
- ゲームパッド・タッチ操作・文字の大きさ・色覚への配慮、英語などへの翻訳
- 日本語入力（今はボタンで名前を付け足す方式）

シナリオ仕立てのチュートリアル

- 例：「墜落した技術者」が壊れた船を直して月の街へ帰るまで
- 章ごとに1つの目的：歩く → 掘る → 直す → 動かす → 制御する → 飛ぶ → 届ける
- 看板の説明は、無線で話しかけてくる案内役の会話と目標表示に置き換える
- 終わったあとは街の「依頼」（運送・建設・都市への助言）で目標が続く
- 技術ツリーで機器が少しずつ解放される

大事にしたい点：見た目はくっきりさせても「ブロックが曲がる・ちぎれる」手触りは残す。形の作り方（表示用メッシュ）を切り替えるだけで、物理の仕組みは今のまま使える。

改善案の優先度 — 効き目と手間



色の意味

柱1 軽量化　柱2 マルチプレイ　柱3 ゲーム性

● フェーズ2 の前半で着手

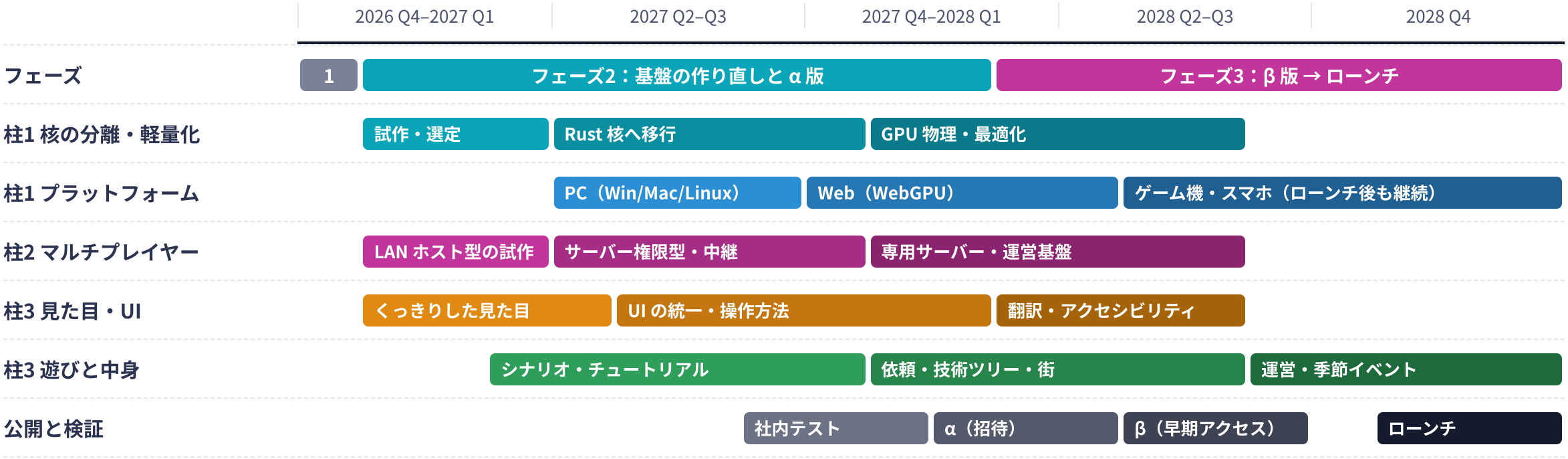
- くっきりした見た目（面ごとの UV・テクスチャ）
- UI のヒント表示とデザインの統一
- 剛体を標準にした軽量化
- LAN のホスト型でのマルチプレイ試作

● フェーズ2 の本命

- 核の分離（Rust → ネイティブ／WASM）
- サーバー権限型の同期と中継
- シナリオ仕立てのチュートリアル

フェーズ3（ローンチ）までの全体像

四半期単位の目安。少人数（プログラマー 2〜3 名＋アート・UI 1〜2 名）を想定し、各フェーズの終わりに「進んでよいか」の判断基準を置く。



※ 日程は体制と判断基準の結果で前後する前提の目安。フェーズ1（プロトタイプ）は 2026 年 10 月に完了。

フェーズ2 前半（〜2027 Q1）— 試作と選定、すぐ効く改善

核の試作と選定

- 物理の芯（ボクセルの形合わせ・結合・関節つき剛体）を Rust で試作し、今の Python 版と結果を比べる
- WASM でブラウザでも同じ結果になるか確かめる
- 決まった順・決まった刻みの計算にして、同じ入力なら同じ結果を出す（再現テスト）
- 描画エンジンの候補を、同じ街の場面で比べる

マルチプレイの試作

- 今の Python 版の上で LAN のホスト型を試す（2〜4 人）
- 送る内容：部分ごとの剛体の姿勢＋変形の差分＋機器の状態
- 通信量・遅延・ずれの測り方を決める
- セーブを「ワールド」と「プレイヤー」に分ける

見た目と UI（今の版にも反映）

- 面ごとの UV とくっきりしたテクスチャ集
- 角の丸みを小さく、縁の線と軽い陰
- 画面下のヒント表示、デザインの決まりを作って主要な画面から適用
- シナリオの台本と、章ごとの目的の設計

この期間の終わりの判断基準

Rust 核の物理が Python 版と同じ動きで **5 倍以上速い**

LAN で **4 人**が同じワールドで 30 分遊んでも、ずれや切断がない

初めての人の **8 割**が、説明なしでブロックの形を見分けられる

フェーズ2 後半（2027 Q2〜Q3）— 新しい基盤への移行と α 版

核の移行と PC 版

- 物理・機器・世界・街・経済を Rust 核へ順に移す（Python 版は比較用に残す）
- ゲーム内スクリプト：Python 風の言語のサンドボックスを用意し、今のスクリプトとブロックプログラムを動かす
- Windows / Mac / Linux で同じビルド。ワンクリック配布は今の仕組みを引き継ぐ

マルチプレイの本実装

- サーバー権限型：ホスト型と中継（NAT 越え）を同じ作りで
- 近くの構築物だけ細かく送る（関心範囲）、遠くはレールのまま
- スクリプトと無線はサーバー側で実行・仲介

遊びの中身

- シナリオ・チュートリアル（第1章〜第3章：墜落・修理・最初の飛行）
- UI の統一を全画面へ。ゲームパッドの操作
- 街の「依頼」の試作（運送・建設）

α 版（招待制・2027 Q3 末）の判断基準

- 中くらいの PC で、街の中・構築物 50 個で **60 fps**
- ホスト型で **8 人**、中継経由で遅延 150 ms でも遊べる
- チュートリアルの第1章を **7 割**の人が最後まで進める
- ゲームが落ちる回数：1 時間あたり 0.1 回未満

PART 3 ／ フェーズ3

フェーズ3（2027 Q4〜2028 Q4）— β 版からローンチへ

仕上げと広げる

- 物理の GPU 化（WebGPU / Vulkan / Metal）で大きな構築物を軽く
- Web 版（ブラウザで体験版）
- ゲーム機・スマホ版は表示側を汎用ゲームエンジンで。ローンチ後の追加も視野

オンラインの運営

- 専用サーバー（公開ワールド・街と経済の共有）
- アカウント・フレンド・招待、不正対策、通報
- 構築物と MOD の共有（ワークショップ）、作者の表示
- 監視・バックアップ・障害時の手順

遊びの中身と品質

- シナリオの残りの章、依頼・技術ツリー・都市の思惑の調整
- 翻訳（英語・中国語など）とアクセシビリティ
- β（早期アクセス）でバランス・負荷・不具合を集めて直す
- ローンチ後：季節イベント・新しい天体・機器の MOD 公式セット

ローンチの判断基準（2028 Q4 目安）

β で遊んだ人の **翌週も遊ぶ割合 40% 以上**

専用サーバー1台で **32 人**、平均 30 fps 以上（クライアント）

重大な不具合（セーブ破損・進行不能）**ゼロ**

PC 3 種と Web で **同じワールドに参加**できる

リスクと対策、そして次の一手

リスク	対策
書き直しが長引く	物理の芯から順に移し、Python 版と同じ結果になるかを自動テストで常に比べる。移せていない部分は Python 版のまま遊べる状態を保つ
やわらかい物理の同期が重い	関節つき剛体を標準にし、ブロックごとの計算は壊れる瞬間と近くだけ。送るのは部分の姿勢と差分
スクリプトの安全性	サーバー側のサンドボックス（時間・メモリの上限）、通信量の上限。無線の情報の大きさ制限は今の版にもある
見た目を変えると手触りが失われる	表示用の形だけを切り替え、物理は今のまま。くっきり表示と今の丸い表示を設定で選べるようにして比べる
プラットフォームごとの対応が重い	核を共通にし、表示側は PC → Web → ゲーム機・スマホの順に。ゲーム機・スマホはローンチ後でもよい

次の 4 週間でやること

- Rust による物理の芯の試作（形合わせ・結合・関節つき剛体）と、Python 版との比較テスト
- 面ごとの UV とくっきりテクスチャを今の版で試し、見え方を比べる
- LAN のホスト型マルチプレイの最小試作（2 人で同じ構築物を動かす）
- シナリオの台本（第1章）と、画面下のヒント表示

フェーズ1 の成果物

Python 版一式（ゲーム・MOD の例・チュートリアル）、Windows 用ワンクリックパッケージ、GPU シェーダー、README（仕組みと API の説明）

まとめ　フェーズ1 で「物理 × 機器 × プログラム」の遊びの芯と、街・経済・無線まで広がる世界を Python 版で形にした。フェーズ2 では **核の分離・マルチプレイ・くっきりした見た目と物語** に取り組み、α 版で手応えを確かめてから、フェーズ3 で β を経てローンチを目指す。